STARK ENGINEERING LAB

MULTI-HAZARD DETECTION AND EMERGENCY RESPONSE SYSTEM

Final implementation report • Arduino Uno • Tinkercad Circuits

Project purpose
A low-cost educational prototype that monitors an MQ-135 air-quality sensor, a digital flame input and PIR motion context. When the gas reading reaches the danger threshold or the flame input is active, the system indicates danger using a red LED, buzzer and fan output. A 16×2 I²C LCD and Serial Monitor display status and readings.

Documentation basis
This report uses the final Arduino sketch supplied by the project owner and the owner's confirmations: MQ-135 is the gas sensor; D8 is connected to the TMP36; the fan is connected through a transistor driver and operates; push buttons are not used; green indication remains on continuously while red indication is active only during an alarm.

Important engineering note
The supplied sketch reads D8 as a digital flame input. It does not read the TMP36 as a temperature sensor. Therefore the TMP36 is not functionally integrated in this exact final code. This discrepancy is documented openly and must be corrected before claiming temperature monitoring.

Prepared as an educational build and implementation guide

1. Project overview

The Multi-Hazard Detection and Emergency Response System is an Arduino-based prototype intended to demonstrate how multiple inputs can be monitored and how an emergency response can be triggered. It combines sensor acquisition, threshold-based decision-making, local alerts, a display and an actuator output.

The final sketch deliberately uses a simple decision rule: the MQ-135 danger threshold OR the digital flame input can activate the alarm. PIR motion is shown as contextual information only; it does not trigger the alarm. The green LED is a continuously illuminated power/ready indicator, while the red LED is illuminated only during an alarm.

Project objectives

• Read the MQ-135 module's analog output through Arduino A0.
• Classify the raw gas-sensor reading as SAFE, MILD or DANGER.
• Read a digital flame signal on D8, with LOW interpreted as flame detected.
• Read PIR motion on D2 and show presence as Y/N without changing alarm state.
• Activate the red LED, buzzer and transistor-driven fan output when gas is in DANGER or flame is detected.
• Show the overall status and gas reading on a 16×2 I²C LCD and print diagnostic information to Serial Monitor.

What this project is—and is not

It is a learning prototype for sensor interfacing and alert logic. It is not a certified fire alarm, gas-leak detector, life-safety controller or building evacuation system. The MQ-135 raw ADC threshold is not a calibrated gas concentration, and the simulation cannot establish real-world detection accuracy.

2. Final system workflow

1. Power-up: Arduino configures the I/O pins, starts Serial communication, initialises the I²C LCD, displays “HAZARD MONITOR” and clears the display.
2. Continuous monitoring: every loop reads PIR on D2, MQ-135 analog value on A0 and the digital input on D8.
3. Gas classification: a reading below 60 is SAFE; 60–99 is MILD; 100 or above is DANGER. These are raw ADC counts, not ppm values.
4. Flame decision: D8 LOW is treated as flame detected; HIGH is treated as no flame.
5. Alarm decision: alarm = gasDanger OR flameDanger. PIR is not part of this decision.
6. Response: during alarm, red LED D10 turns on, fan control D9 goes HIGH and buzzer D13 sounds at 1 kHz. Otherwise these outputs are switched off.
7. Indication: green LED D11 is set HIGH every loop and therefore remains on. LCD and Serial Monitor show the status and readings.
8. Repeat: a 500 ms delay sets the approximate update interval.

Decision logic

Condition	LCD overall status	Red LED / buzzer / fan
Gas < 60 and no flame	SAFE	OFF
Gas 60–99 and no flame	MILD	OFF
Gas ≥ 100 and no flame	DANGER	ON
Flame input LOW (regardless of gas)	DANGER	ON
PIR detects a person	PIR field becomes Y; status unchanged	No effect

3. Components used

Component	Qty.	Role / notes
Arduino Uno R3	1	Main controller; reads sensors and drives indicators.
MQ-135 air-quality sensor module	1	Analog air-quality/gas response to A0. Module board is assumed to expose AO, DO, VCC and GND; use AO.
TMP36 temperature sensor	1	Shown/confirmed on D8 in the user's circuit description, but not correctly used by this final sketch. See critical note in Section 5.
PIR motion sensor module	1	Digital OUT to D2; presence is display context only.
Digital flame sensor module	1 only if D8 is intended as flame input	The code requires a digital flame signal on D8. This conflicts with the user's confirmation that TMP36 is connected to D8. Do not connect both outputs together.
16×2 I²C LCD module	1	I²C address set to 0x27; displays status and gas/PIR values.
Green LED	1	D11; continuously ON in this sketch.
Red LED	1	D10; ON only while alarm is active.
220–330 Ω LED resistors	2	One in series with each LED.
Buzzer (compatible with tone())	1	Positive/control connection to D13; common GND. A passive piezo is suitable for tone().
DC fan / small DC motor	1	Switched by an external transistor driver, not directly from Arduino I/O.
NPN transistor or suitable logic-level MOSFET	1	Fan low-side switch; exact part depends on fan current.
Base/gate resistor	1	Typical starting value 1 kΩ for a small NPN transistor; select for the actual transistor and fan current.
Flyback diode	1	Across a brushed DC motor/fan, reverse-biased during normal operation; choose current rating for the load.
External motor supply	1	Voltage matched to the fan; its GND must be common with Arduino GND.
Breadboard and jumper wires	As required	Prototyping and interconnections.
USB / regulated 5 V logic supply	1	Arduino and sensor/LCD logic supply as appropriate.

The final sketch does not use push buttons. They are intentionally excluded from the final component/function list. The TMP36 is physically present according to the user's clarification, but it is not functionally measured by this sketch.

4. Pin assignment and connection map

Arduino pin	Connects to	Connection details
A0	MQ-135 AO	Module AO → A0. Module VCC → supply specified for that module; GND → common GND.
D2	PIR OUT	PIR VCC → module-rated supply; GND → common GND; OUT → D2.
D8	Digital flame sensor DO (as required by code)	DO → D8. Code treats LOW as flame detected. This is NOT a valid TMP36 temperature connection.
D11	Green LED	D11 → 220–330 Ω resistor → LED anode; LED cathode → GND.
D10	Red LED	D10 → 220–330 Ω resistor → LED anode; LED cathode → GND.
D13	Buzzer	Buzzer + / signal → D13; buzzer – → GND, subject to buzzer module requirements.
D9	Fan driver control	D9 → base resistor → NPN base (or suitable MOSFET gate circuit). Do not drive fan directly from D9.

Arduino pin	Connects to	Connection details
A4 (SDA)	LCD SDA	I²C data line.
A5 (SCL)	LCD SCL	I²C clock line.
5V / suitable supply	Sensor and LCD VCC	Check each module's rated supply and logic compatibility.
GND	All low-voltage module grounds	Common ground required, including external fan supply negative.

5. Critical circuit/code consistency check

The user confirmed that the TMP36 is connected to D8. The final sketch declares tsensor = 8 and executes flame_pin = digitalRead(tsensor), followed by flameDanger = (flame_pin == LOW). This means the program interprets D8 as a digital flame-detector output. A TMP36 is an analog temperature sensor: its output is a voltage that changes with temperature and must be read using analogRead() on an analog input (for example A1), followed by the TMP36 conversion formula.

Consequences

• With the current exact sketch, the TMP36 temperature is not measured.
• A TMP36 output connected to D8 may be read only as a digital HIGH/LOW level; this cannot provide a meaningful temperature value.
• The sketch may falsely interpret the TMP36 voltage crossing the digital input threshold as a flame event.
• The code contains no temperature threshold, no temperature display and no temperature-based alarm.
• The code requires a digital flame sensor output on D8 if flame detection is to work as intended.

To retain the code exactly as requested, the circuit must provide a digital flame-sensor DO signal on D8. The TMP36 can remain physically installed only if it is disconnected from D8 and left unused, or connected to a separate analog pin but not treated as an active feature. If temperature monitoring is required, the code and pin map must be revised in a later version. Do not join the TMP36 output and flame sensor output on the same Arduino pin.

6. Fan driver wiring — physical implementation

The fan must not be powered directly from Arduino D9. D9 is a control signal only. A typical low-side NPN transistor arrangement is:

Connection point	Wiring
Arduino D9	Through approximately 1 kΩ base resistor to NPN transistor base (verify transistor pinout).
NPN emitter	To common GND.
NPN collector	To fan negative terminal.
Fan positive terminal	To the positive terminal of the fan's correctly rated external supply.
Flyback diode	Across fan terminals: cathode/banded end to fan positive; anode to fan negative/transistor collector.
External supply negative	To Arduino GND (common ground).

The transistor, diode and supply must be selected for the actual fan voltage and stall/startup current. If the fan draws more current than the transistor can safely switch, use a properly rated logic-level MOSFET or driver module. Do not assume a small signal transistor is adequate. The diode is required for inductive brushed DC motor loads; observe polarity carefully.

7. LCD and sensor wiring notes

I²C LCD: connect VCC to the LCD module's rated logic supply, GND to common GND, SDA to Arduino A4 and SCL to Arduino A5. The sketch expects I²C address 0x27 and a 16-column, 2-row display. If the LCD stays blank, verify supply, contrast/backpack soldering, address and I²C wiring; do not change the supplied code unless the actual address/library setup requires it.

MQ-135: connect the module's analog output AO to A0. The digital output DO is not used by this sketch. Power the particular module according to its datasheet/module board specification. Many MQ-135 boards use a 5 V heater and require warm-up and calibration. The raw analog value depends on the module, supply, warm-up, sensor history, environment and ADC reference.

PIR: connect OUT to D2 and use a module supply appropriate to its specification. The code uses INPUT mode and expects a defined digital output from the PIR module. PIR detects changes in infrared radiation associated with movement; it does not identify a person or prove occupancy with certainty.

8. Final Arduino code — preserved exactly as supplied

The following is the user's requested final sketch. It is reproduced without edits. The technical inconsistency involving D8/TMP36 is explained in Section 5; preserving the code here does not certify that every connected component is used correctly.

```
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

int greenled = 11;
int buzzer = 13;
int sensor = A0;      // gas/smoke sensor
int midThre = 60;     // ABOVE this = MILD   (tune after testing)
int thre = 100;        // ABOVE this = DANGER
int psensor = 2;
int tsensor = 8;
int flame_pin = HIGH;

int redled = 10;
int fanpin = 9;

void setup() {
  pinMode(buzzer, OUTPUT);
  pinMode(greenled, OUTPUT);
  pinMode(tsensor, INPUT);
  pinMode(psensor, INPUT);
  pinMode(sensor, INPUT);
  pinMode(redled, OUTPUT);
  pinMode(fanpin, OUTPUT);
  digitalWrite(fanpin, LOW);

  Serial.begin(9600);

  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("HAZARD MONITOR");
  delay(1000);
  lcd.clear();
}

void loop()
{
  digitalWrite(greenled, HIGH);

  int read = digitalRead(psensor);
  int analogValue = analogRead(sensor);
  flame_pin = digitalRead(tsensor);

  String gasStatus;
  bool gasDanger = false;
  if (analogValue >= thre) {
    gasStatus = "DANGER";
    gasDanger = true;
  } else if (analogValue >= midThre) {
    gasStatus = "MILD";
  } else {
    gasStatus = "SAFE";
  }

  bool flameDanger = (flame_pin == LOW);
  bool personPresent = (read == HIGH);   // PIR = context only, not an alarm trigger

  // ---- only gas and flame can set off the alarm ----
  bool alarm = gasDanger || flameDanger;

  digitalWrite(redled, alarm ? HIGH : LOW);
  digitalWrite(fanpin, alarm ? HIGH : LOW);

  if (alarm) tone(buzzer, 1000);
  else noTone(buzzer);

  String overall = alarm ? "DANGER" : gasStatus;

  lcd.setCursor(0, 0);
  lcd.print("STATUS: ");
  lcd.print(overall);
  lcd.print("   ");
  lcd.setCursor(0, 1);
  lcd.print("GAS:");
  lcd.print(analogValue);
  lcd.print(" P:");
  lcd.print(personPresent ? "Y" : "N");
  lcd.print("     ");

  Serial.print("GAS:"); Serial.print(analogValue);
```

```
  Serial.print(" FLAME:"); Serial.print(flameDanger ? "DANGER" : "ok");
  Serial.print(" PIR:"); Serial.print(personPresent ? "YES" : "no");
  Serial.print(" OVERALL:"); Serial.println(overall);

  delay(500);
}
```

9. Explanation of the final code

Library and LCD object: LiquidCrystal_I2C.h provides control of the I²C LCD. The object is configured for address 0x27 and a 16×2 display.

Pin and threshold variables: greenled=11, buzzer=13, sensor=A0, psensor=2, tsensor=8, redled=10 and fanpin=9 define the hardware interface. midThre=60 and thre=100 are raw analog thresholds for the MQ-135 reading.

setup(): configures outputs and inputs, forces the fan output LOW initially, starts Serial at 9600 baud, initialises the LCD, displays the startup message and clears the screen.

Sensor acquisition: digitalRead(psensor) obtains the PIR state; analogRead(sensor) obtains the MQ-135 ADC reading; digitalRead(tsensor) obtains a digital input that the code calls flame_pin.

Gas classification: values ≥100 are DANGER and set gasDanger=true; values ≥60 and below 100 are MILD; values below 60 are SAFE. These numbers are not ppm or a validated air-quality index.

Flame and PIR logic: LOW on D8 is interpreted as flame danger. HIGH on D2 is interpreted as person present. PIR is display-only and does not activate the alarm.

Alarm response: alarm is true if gasDanger OR flameDanger. The red LED and fan output are set HIGH during alarm. tone(buzzer,1000) produces a 1 kHz tone; noTone() stops it when there is no alarm.

Overall status: if alarm is active, LCD status becomes DANGER; otherwise it displays the gas classification SAFE or MILD. Therefore a MILD gas condition does not turn on the red LED, buzzer or fan.

Display and serial output: the LCD shows overall status, raw gas ADC reading and PIR Y/N. Serial Monitor prints gas value, flame interpretation, PIR state and overall status.

Timing: delay(500) gives an approximate two-updates-per-second loop. It is simple but blocks the processor during the delay.

Green LED: digitalWrite(greenled,HIGH) runs every loop and the code never sets it LOW. It is therefore continuously illuminated, including during an alarm. It is a power/ready indicator, not a SAFE-only indicator.

10. Operating-state table

Input condition	LCD status	Green D11	Red D10	Buzzer D13	Fan D9
Gas < 60; D8 HIGH	SAFE	ON	OFF	OFF	OFF
Gas 60–99; D8 HIGH	MILD	ON	OFF	OFF	OFF
Gas ≥ 100; D8 HIGH	DANGER	ON	ON	ON	ON
D8 LOW; any gas reading	DANGER	ON	ON	ON	ON
PIR HIGH; no gas/flame danger	Gas status unchanged; P:Y	ON	OFF	OFF	OFF

11. Testing and validation record

The project owner reports that the Tinkercad circuit and simulation are completed and that the fan operates through a transistor driver. This report records that owner-provided status. The author of this report could not independently inspect a fresh Tinkercad project or run the simulation, because no current share link or circuit screenshot was available at report preparation time.

Test	Expected behaviour from code	Record / pass criterion
Startup	LCD shows HAZARD MONITOR briefly, then clears.	Confirm LCD address, wiring and visible text.
MQ-135 below 60	LCD SAFE; alarm outputs OFF.	Record raw ADC value after stable warm-up.
MQ-135 60–99	LCD MILD; alarm outputs remain OFF.	Verify threshold transition.
MQ-135 ≥100	LCD DANGER; red, buzzer and fan ON.	Verify all three outputs and safe motor switching.
D8 HIGH	No flame danger from this input.	Test with a digital flame module output, not TMP36.
D8 LOW	DANGER; red, buzzer and fan ON.	Verify flame input polarity.
PIR LOW/HIGH	P:N / P:Y; no alarm change by itself.	Confirm PIR is contextual only.
Return to safe	Red, buzzer and fan turn OFF; green stays ON.	Verify output reset after inputs return safe.

12. Limitations — honest engineering scope

MQ-135 calibration: the sketch uses raw 10-bit ADC counts (0–1023). Thresholds 60 and 100 are provisional values for a simulation/demo and cannot be translated directly into ppm, toxic-gas concentration or a validated air-quality score. A real MQ-135 requires correct heater supply, warm-up/burn-in as specified by its datasheet, calibration in known reference conditions, compensation/interpretation appropriate to the target gases and repeatable testing. MQ-135 is a broad-response sensor; it does not uniquely identify every gas.

Temperature is not monitored by this final sketch: although a TMP36 is reported connected to D8, the program performs digitalRead(D8). It has no analog temperature conversion or temperature threshold. Temperature detection must not be claimed until the circuit uses an analog pin and the sketch is updated and tested.

Flame detection dependency: code assumes a digital flame sensor output on D8 and assumes LOW means flame. Sensor modules may have adjustable comparators and different output polarity. Verify the exact module and test its DO output. A TMP36 cannot serve as this digital flame input.

PIR limitations: PIR detects motion-related changes, not identity, exact distance or reliable presence in all conditions. It is not an alarm trigger in this sketch.

Fan response: fan activation is a demonstration actuator. A fan may spread smoke, flame or hazardous vapours; in a real fire/gas event, automatically switching on an ordinary fan can be unsafe. A real installation needs a hazard-specific, professionally engineered ventilation strategy and appropriate certified equipment.

Software limitations: delay(500) blocks the loop; String objects can be undesirable in long-running memory-constrained embedded systems; no sensor fault detection, watchdog strategy, event logging, alarm latching, battery backup, redundancy or communications are implemented. There is no fail-safe response if a sensor wire disconnects or a sensor fails.

Simulation versus reality: Tinkercad models are simplified. A successful simulation does not prove sensor calibration, response time, environmental robustness, electrical safety, long-term reliability or compliance with fire/building codes.

13. How to implement the project physically

Build the physical prototype in stages. Do not begin with a live smoke, flame or gas test. First verify the controller, display and outputs with safe manual input simulation; then integrate each sensor according to its datasheet.

Stage A — power and controller

1. Use an Arduino Uno and a regulated supply appropriate to the board. Avoid powering the fan from the Uno 5 V pin.
2. Establish a common ground between Arduino, sensor modules, LCD and external fan driver supply.
3. Check polarity and voltage with a multimeter before inserting modules.

Stage B — display and indicators

4. Wire the I²C LCD: VCC, GND, SDA→A4, SCL→A5. Confirm address 0x27 using an I²C scanner if needed; the current code expects 0x27.
5. Wire each LED in series with its own 220–330 Ω resistor: D11 for green and D10 for red; LED cathodes return to GND.
6. Connect a compatible buzzer to D13 and GND. If the buzzer module draws more than a GPIO can safely supply, use a transistor driver.

Stage C — sensors

7. MQ-135 module: connect AO→A0, VCC and GND as specified by the module. Allow the sensor to warm up and do not use the simulation thresholds as calibrated real-world limits.
8. PIR: connect OUT→D2, VCC and GND per the PIR datasheet.
9. Digital flame sensor: connect DO→D8 only if D8 is to perform the flame function in this exact code. Test whether its active state is LOW.
10. TMP36: do not connect its output to D8 expecting temperature measurement. For actual temperature monitoring, connect TMP36 output to a separate analog input (e.g. A1), power it correctly, and implement analogRead plus the TMP36 conversion in a revised sketch. This would be a code revision, not the exact code printed in this report.

Stage D — fan driver

11. Identify the fan's rated voltage and running/stall current. Select a transistor or MOSFET with suitable voltage/current and thermal ratings.
12. Wire the fan through the transistor low-side switch and add the flyback diode across the motor terminals with correct polarity.
13. Use an external supply rated for the fan. Join its negative terminal to Arduino GND. Connect Arduino D9 only to the transistor control input through the appropriate resistor/driver circuit.
14. Test the fan switching without hazardous gases or flame. Confirm the transistor does not overheat and the Arduino does not reset.

Stage E — calibration and validation

15. Record clean-air baseline readings after the MQ-135 has reached stable operating conditions. Use controlled, safe, known test conditions and follow the sensor datasheet. Do not expose the sensor to open flame, concentrated gas, toxic vapours or improvised smoke sources.
16. Validate each input separately, then test combined conditions and recovery to normal.
17. For any real deployment, use certified detectors and professionally designed alarm/ventilation equipment; this educational prototype must not be the sole safety system.

14. Further improvements (future version, not part of the final code)

- Correctly integrate TMP36 using an analog input and temperature thresholds.
- Confirm and document the exact flame sensor model and active polarity.
- Calibrate MQ-135 or use a sensor intended and certified for the target gas; add suitable signal filtering and fault detection.

• Add hysteresis to avoid rapid state changes near thresholds.
• Replace delay() with millis()-based scheduling for more responsive monitoring.
• Add startup sensor checks, sensor-disconnect detection, alarm event logging and a defined fault state.
• Reassess the fan response with a safety professional; automatic ventilation is not universally safe in fire or gas scenarios.

15. Conclusion

The completed educational prototype demonstrates the integration of an MQ-135 analog input, digital inputs, a PIR context signal, I²C display, LEDs, buzzer and a transistor-driven fan output. The final program implements a simple threshold-based gas classification and a combined gas/flame alarm. The green LED is continuously on; the red LED, buzzer and fan are active during a danger alarm. The PIR is informational only.

The report intentionally preserves the supplied final sketch. One important inconsistency remains: the TMP36 is reported connected to D8, while the sketch treats D8 as a digital flame sensor. Consequently, temperature monitoring is not implemented in this final version. The project is suitable as a learning and simulation prototype, but it is not a certified life-safety device and must not replace approved smoke, gas or fire detection systems.

Appendix — quick final pin reference

Function	Pin
MQ-135 analog output	A0
PIR output	D2
Digital flame input expected by code	D8
Red LED	D10
Green LED (always ON)	D11
Buzzer	D13
Fan transistor control	D9
I²C LCD SDA / SCL	A4 / A5